Relatório 19 - Prática: Pipelines de Dados II - Airflow (III)

Matheus Beuren Carvalho

1. Introdução

A seguir serão apresentados os conceitos aprendidos no card 19, onde o conteúdo se refere ao airflow e suas técnicas avançadas. Em sua maioria, o curso ensina a utilização de ferramentas e demonstra sua utilização em um ambiente real.

2. Desenvolvimento

A continuação deste curso se deu inicialmente ao que será feito neste card e dois conceitos muito importantes, que podem ser observados na Tabela 1.

Tabela 1 - Sequential Executor e Local Executor

	SequentialExecutor	LocalExecutor
O que é	Executor default do Airflow, somente uma task por vez	Executa tasks em paralelo
Onde usar	Testes e debug	Produção

Em seguida é apresentado como rodar tasks em paralelo, a partir de parâmetros no “.cfg” como mostra a sequência das Figuras 1 e 2.

Figura 1 - Airflow task rodando

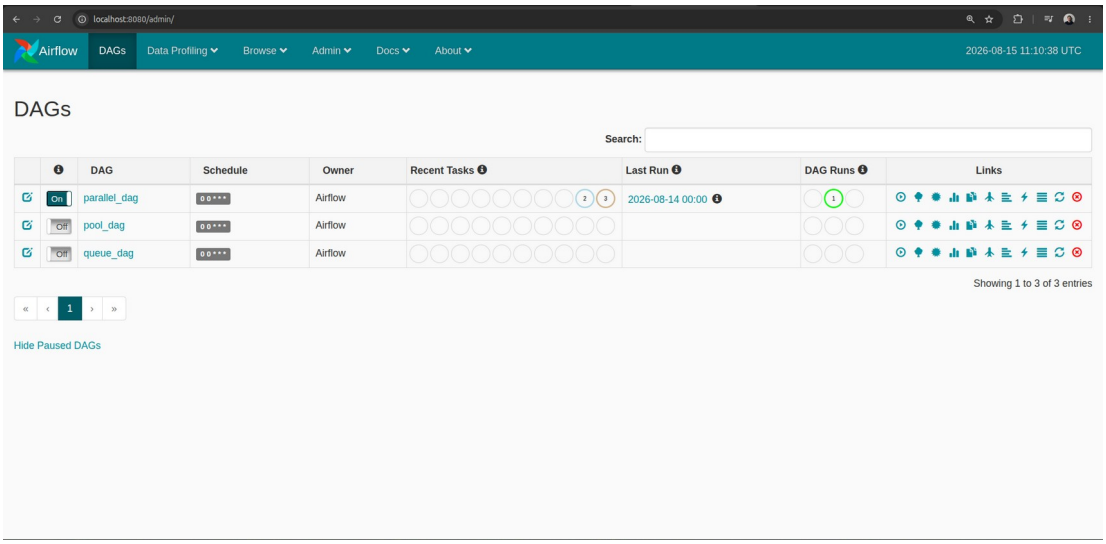
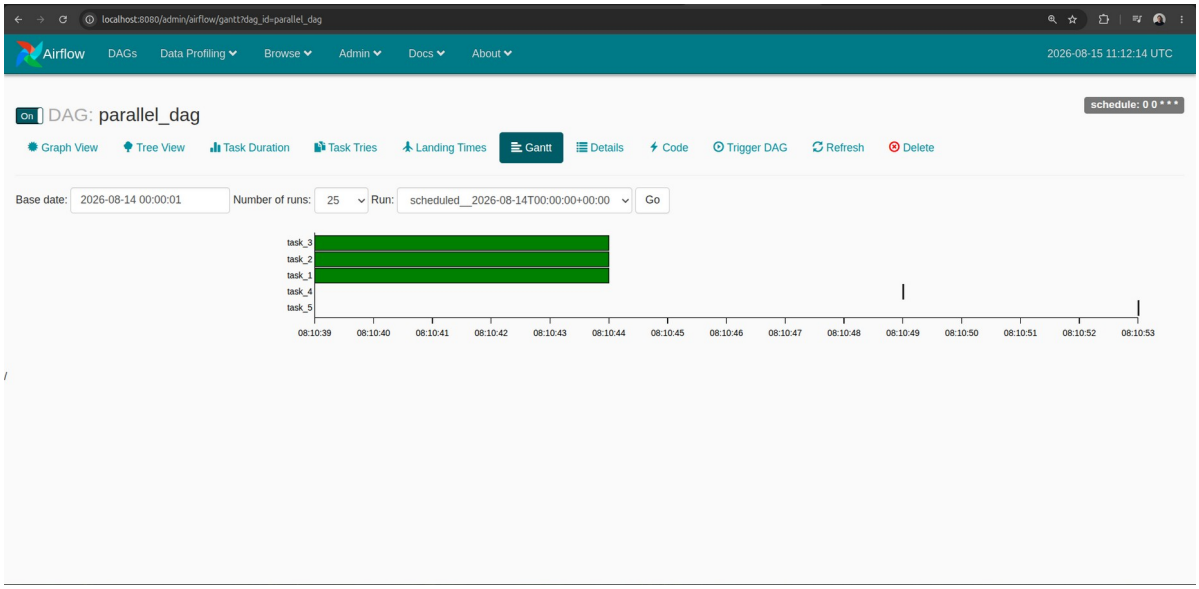
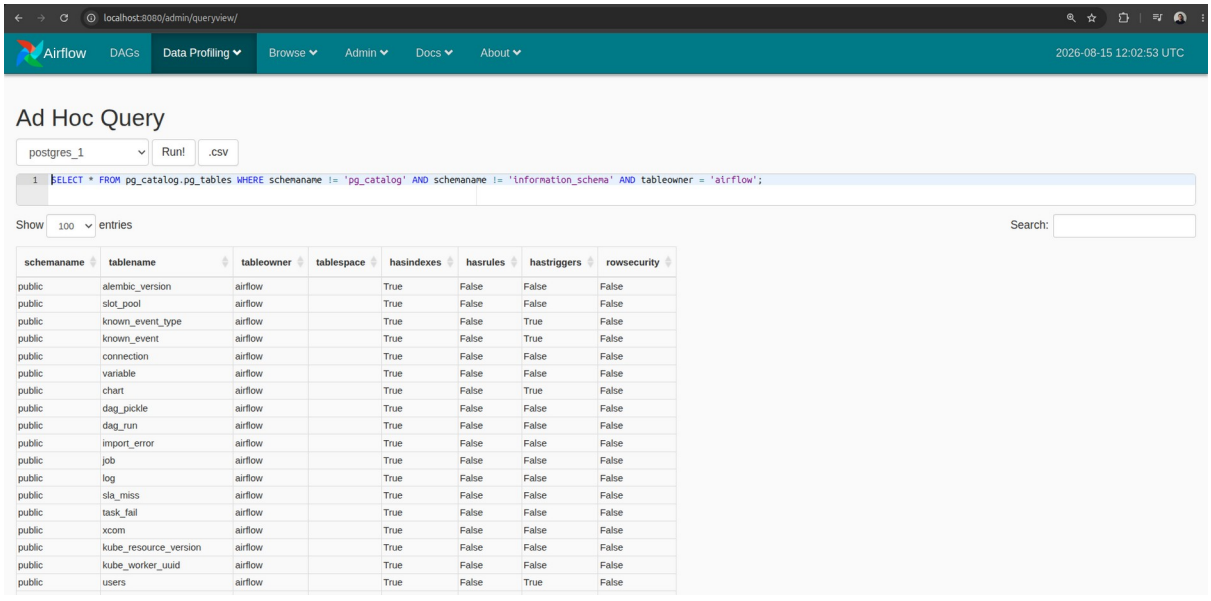


Figura 2 - Airflow task paralelo



Na Figura 3, é mostrado algumas querys de sql para ver os dados do postgresql, onde foi feito uma conexão apontando para o postgresql e depois um configuração que nao nao permite o airflow ver e rodar querys para o banco por questões de segurança.

Figura 3 - Querys no Airflow

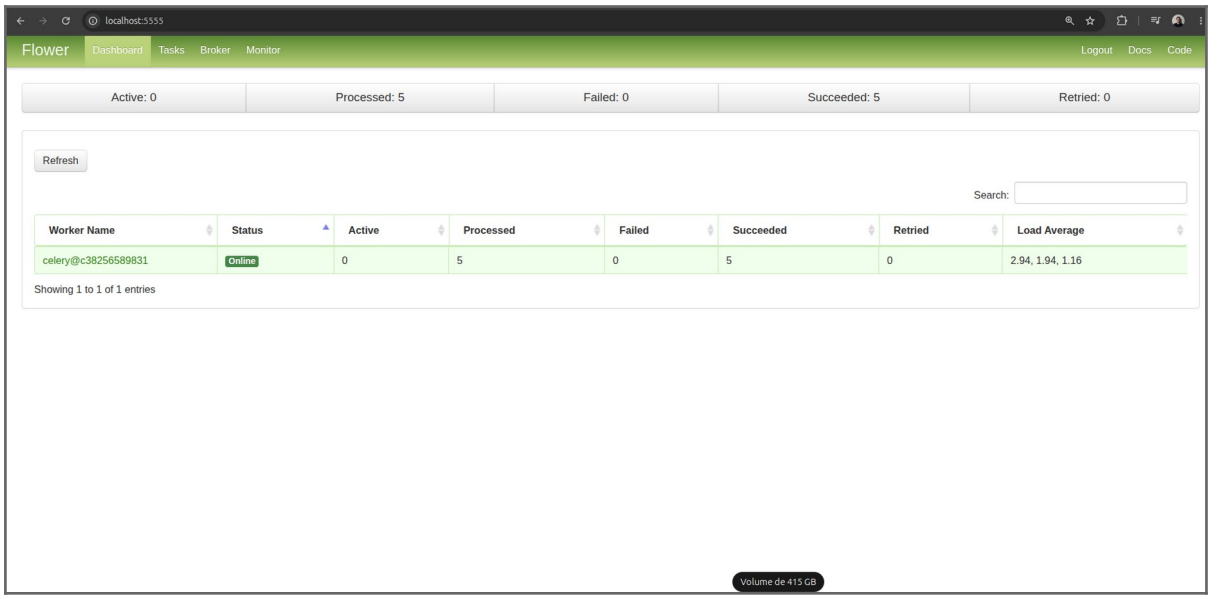


O Celery é uma fila assíncrona que distribui as tasks em diferentes workers do airflow, nisso é necessário de um message broker como por exemplo o RabbitMQ. Utilizando o Celery é possível escalar horizontalmente (adicionar instâncias), após escalar as tasks são distribuídas entre instâncias e executam os

processos. A sua escalabilidade permite que se um worker cair o celery atribui as tasks para outro worker disponível.

Para o monitoramento do celery, o flower é uma ferramenta que monitora tasks e workers do celery, pode ser visto sua interface na Figura 4.

Figura 4 - Flower



Na parte de filas no airflow, o instrutor realizou uma prática onde foi separado as tasks por funcionalidade, por exemplo a task que precisa de mais cpu vai para a fila de cpu que está associada a um worker específico que tenha mais acesso a cpu.

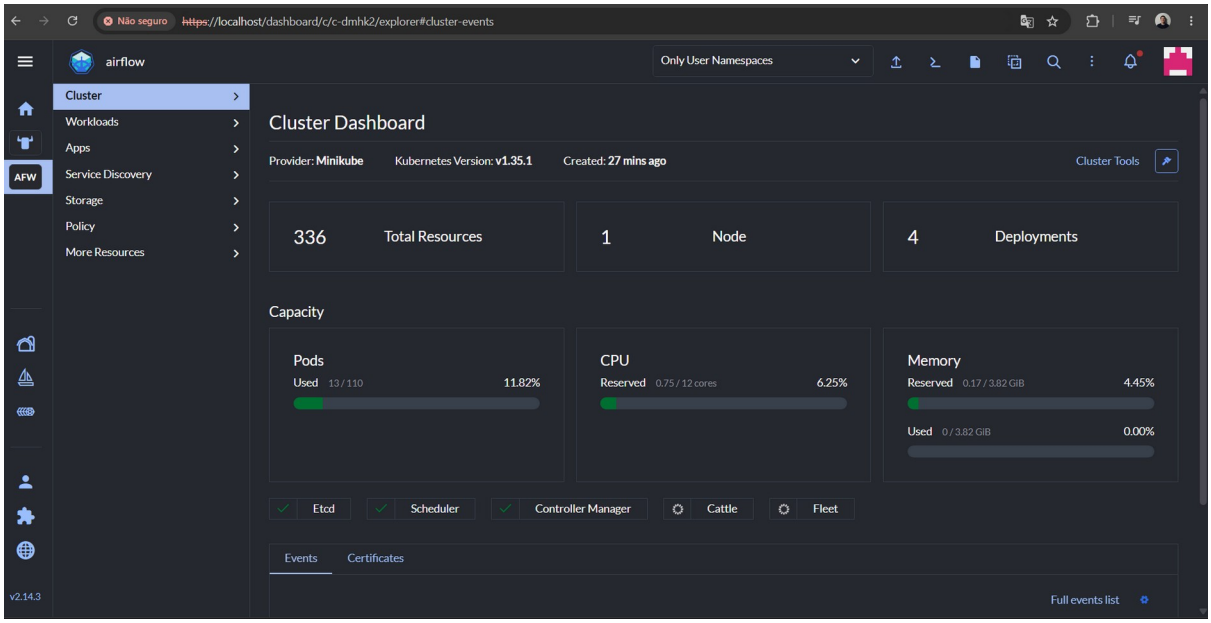
Na execução das tasks de coleta de moedas na api, ainda existe uma diferença em relação ao código do vídeo, atualmente é necessário de uma chave de api para conseguir os dados então tive que fazer algumas modificações no código original.

Nos vídeos o instrutor apresenta o kubernetes, uma ferramenta open-source que é muito famosa pela sua facilidade de escalar objetos kubernetes. Em suma se define como um orquestrador de containers, ferramenta que exige uma curva de aprendizado maior que outras ferramentas, no entanto o instrutor mostra conceitos básicos relacionados ao k8s. Se baseia em um node principal, denominado control plane que tem quatro objetos que operam o kubernetes o apiserver, scheduler, controller-manager e etcd, resumindo servem com as principais objetos que controlam o cluster, tanto o que será feito, quando, banco de dados e mais funções. Os nodes são objetos maiores onde ficam os pods(menor objeto no k8s), onde nos pods são colocados um ou mais containers para assim rodar a aplicação. Além dos objetos já citados, outros objetos como replicaset e deployment são a aplicação de réplicas dos mesmos pods em um número x de réplicas escolhidas que no deployment é o mais utilizado pois oferece atualizações e rollbacks para casos de atualização de imagens em containers, é um dos objetos mais usados. O service redireciona a chamada e faz a conexão de pods ou por meio do CLusterIP que significa a comunicação interna entre pods dentro do cluster.

Diferente do vídeo não utilizei vagrant, mas usei na minha maquina com linux o minikube que simula um cluster kubernetes.

O Rancher é utilizado para evitar comandos via terminal no cluster, ele oferece um gerenciamento de clusters kubernetes via interface web que por sua vez, evita a criação manual de yamls, fazendo conexão com diversas ferramentas. A interface Rancher é apontada na figura 5.

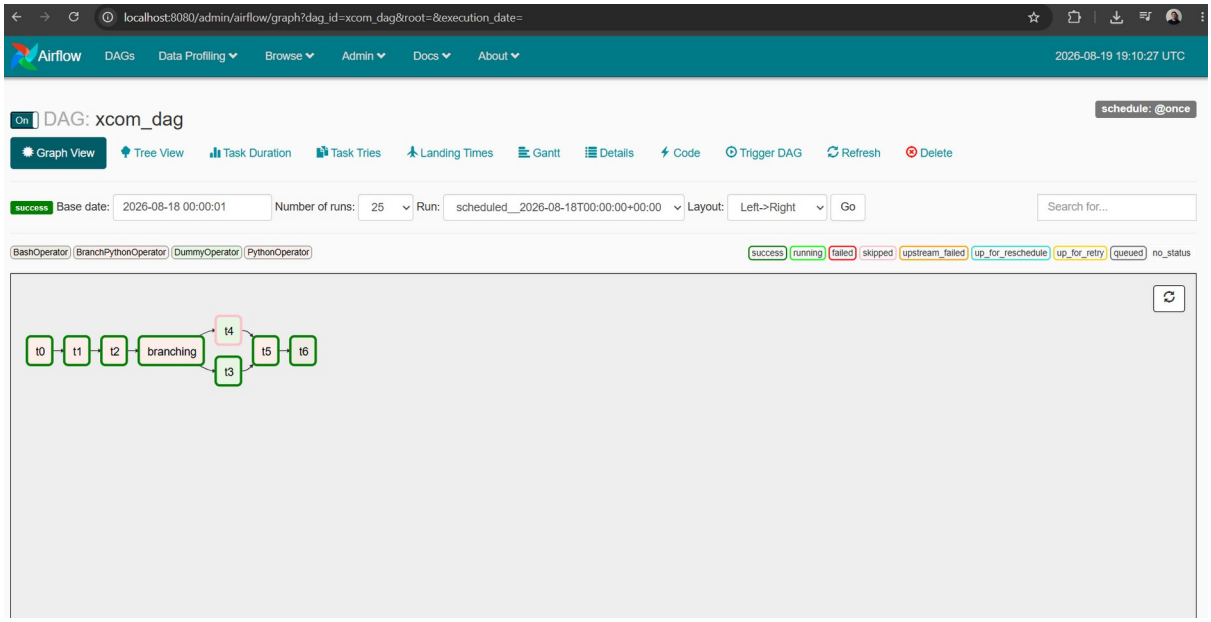
Figura 5 - UI Rancher



Sub-Dags são dags dentro de dags, de preferência dentro de um dag do mesmo fluxo lógico, por exemplo em um dag de coleta de dados onde um sub-dag pode coleta de uma api, outro sub-dag realiza a limpeza, e outro por fim pode jogar para uma predição com machine learning em outro dag. As condições no airflow são baseadas em branches, onde dada uma determinada condição é executado um dag.

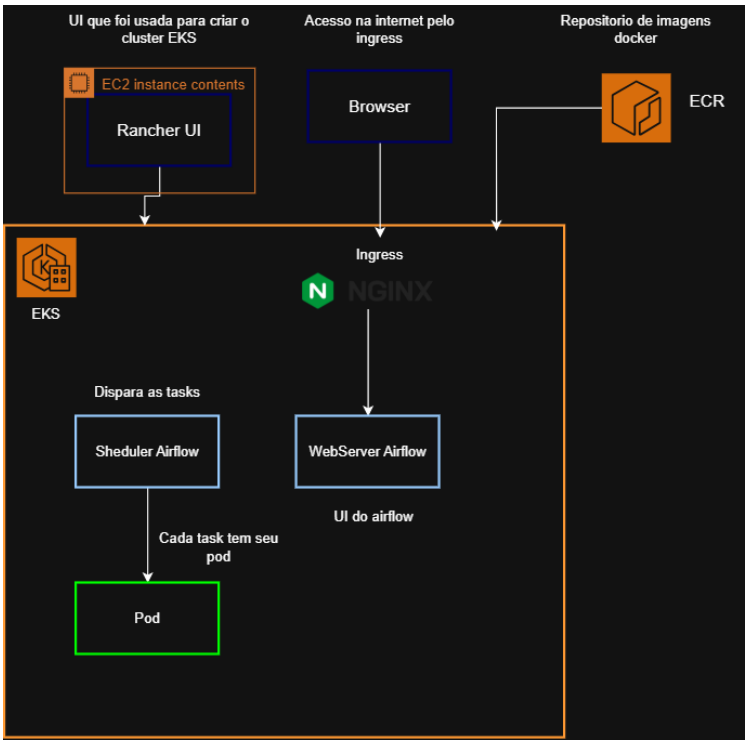
Xcoms na minha opinião é um dos conceitos mais úteis no card, a possibilidade de passar dados entre tasks aumenta e muito a possibilidade e a diversidade de dags, principalmente da parte de engenharia de dados onde ocorre a coleta seja via api ou scrapping, tratamento dos dados e a preparação dos dados para um futuro treino de ML. Na figura 6 mostra a execução do dag xcom.

Figura 6 - Xcom



O AWS EKS, é o serviço da aws que se refere ao kubernetes, nele você cria clusters e coloca suas aplicações. No card em questão foi colocado o airflow para funcionar no kubernetes gerenciado pela aws. A figura 7, mostra a arquitetura usada na seção sete em relação ao airflow no EKS.

Figura 7 - Arquitetura EKS



Por primeiro foi implementado o rancher em uma instância EC2(máquina virtual da aws) para assim provisionar o cluster eks, para isso foi necessário criar um usuário admin para o provisionamento do cluster. No cluster em si, é utilizado

um pod para cada serviço do airflow, onde o scheduler(acionador de tasks) roda uma task por pod, isso ajuda baixo acoplamento onde cada pod possui seu próprio limite memória e cpu que por sua vez facilita a execução de cada task. Em aulas anteriores foi mostrado como separar tasks por necessidade de processamento ou necessidade de memória, assim podendo alocar mais memória ou processadores para alguma task específica.

O monitoramento é um método obrigatório em qualquer ferramenta ou aplicação construída. Automatizar e abstrair logs é essencial para qualquer refatoração, debug ou funcionamento da aplicação. No Airflow não é diferente, o monitoramento se torna essencial pois as operações dependem se foram executadas ou falharam na execução, tudo isso centralizado em uma interface web simples. A aula de monitoramento mostrou os níveis de log que podem ser extraídos do airflow, a figura 8 mostra o 'INFO', um dos niveis do logger.

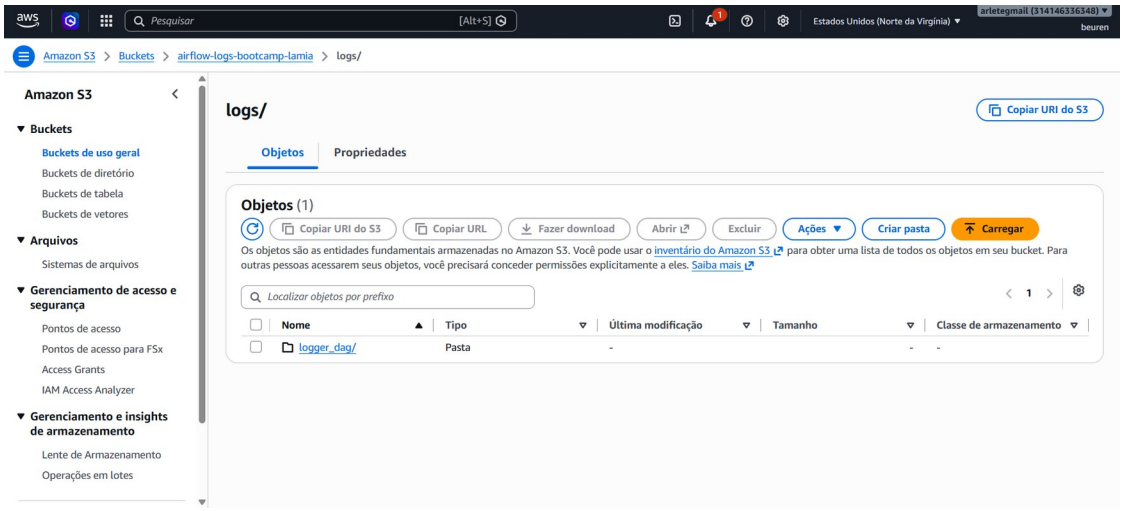
Figura 8 - Log INFO

```
DAG File Processing Stats

File Path                                  PID  Runtime      # DAGs    # Errors  Last Runtime    Last Run
---------------------------------------  -----  ---------  --------  ----------  --------------  ----------
/usr/local/airflow/dags/data_dag.py         19  0.01s             0           0
/usr/local/airflow/dags/logger_dag.py       20  0.00s             0           0
==========================================================================
[2026-08-20 14:11:47,990] {dag_processing.py:1292} INFO - Finding 'running' jobs without a recent heartbeat
[2026-08-20 14:11:47,991] {dag_processing.py:1296} INFO - Failing jobs without heartbeat after 2026-08-20 14:06:47.991049+00:00
[2026-08-20 14:11:58,243] {dag_processing.py:1292} INFO - Finding 'running' jobs without a recent heartbeat
[2026-08-20 14:11:58,244] {dag_processing.py:1296} INFO - Failing jobs without heartbeat after 2026-08-20 14:06:58.244332+00:00
[2026-08-20 14:12:18,675] {dag_processing.py:1292} INFO - Finding 'running' jobs without a recent heartbeat
[2026-08-20 14:12:18,675] {dag_processing.py:1296} INFO - Failing jobs without heartbeat after 2026-08-20 14:07:18.675823+00:00
[2026-08-20 14:12:18,688] {dag_processing.py:1029} INFO -
==========================================================================
DAG File Processing Stats
```

Em seguida foi implementado o dag e seus logs implementados no s3 bucket, um objeto da aws onde é possível guardar objetos, nesse caso os logs do dag.

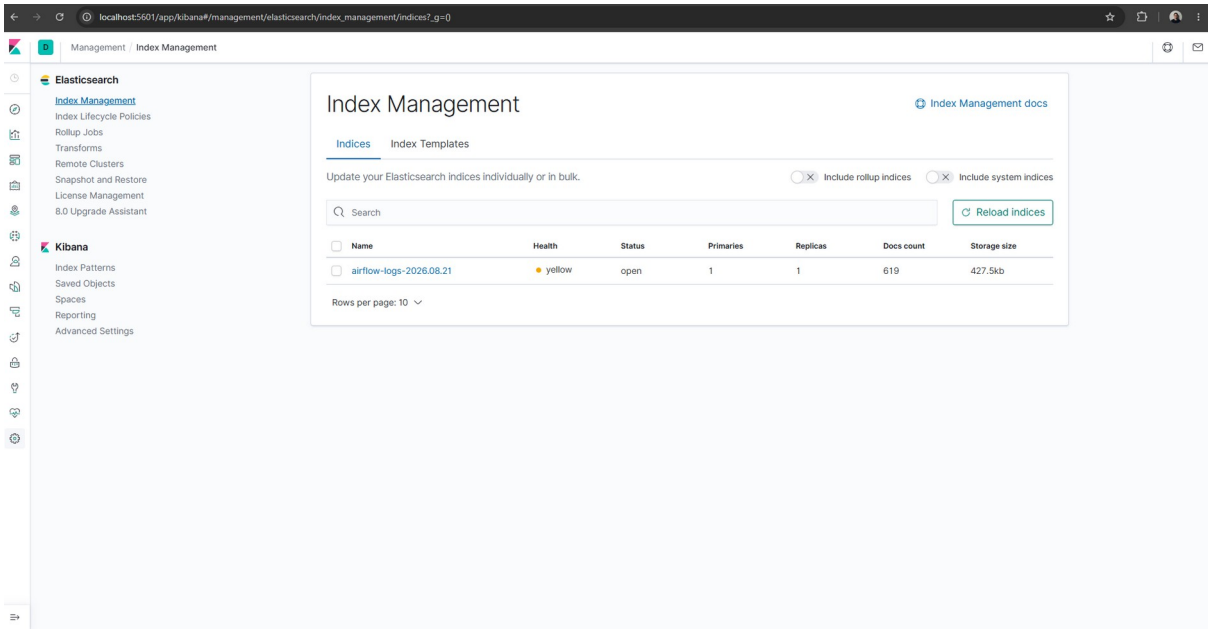
Figura 9 - Logs no s3 bucket



Ainda em questão o monitoramento do airflow, foi utilizado Elasticsearch junto com

outras ferramentas para assim ter uma arquitetura de logs remoto. Assim o kibana foi o receptor desses logs, como mostra a figura 10.

Figura 10 - Logs ElasticSearch



A fernet key é a chave de criptografia que é usada pelo Airflow para proteger dados sensíveis, caso não seja usada, tudo fica exposto em logs e no banco. Também é abordado como é feita a rotatividade desta chave, onde dados que estão na chave antiga nao sera afetados pela troca pois é primeiro usado a chave antiga e depois é criada a chave nova para criptografar

Em seguida é criado um usuário admin para utilizar a RBAC UI que serve para criar usuários e dar permissões para cada usuário. Primeiro é criado via terminal o admin e depois é criado pela UI e assim dando permissoes que ele precisa.

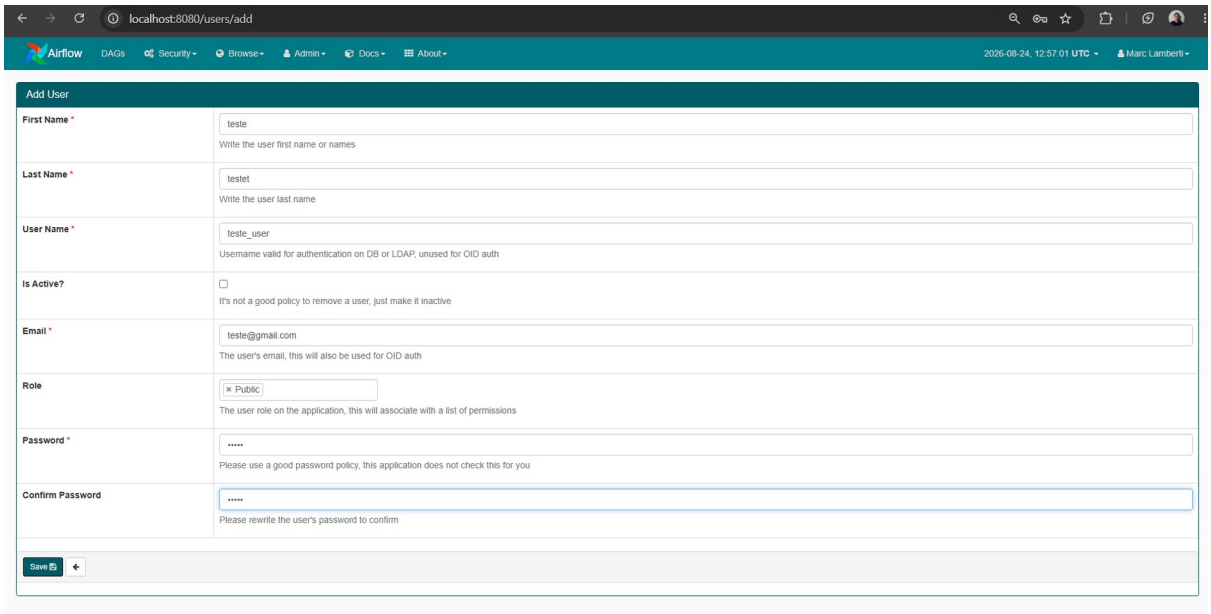
Figura 10 - Criação do usuário via Terminal

```
matheus@LAPTOP-I0U9BL8N:~/LAMIA/Bootcamp-Aprendizado-de-Maquina-LAMIA/card-19/data/airflow-section-9$ docker exec -it 65409b16ec66 /bin/bash
airflow@65409b16ec66:~$ airflow list_users
/usr/local/lib/python3.7/site-packages/flask_sqlalchemy/__init__.py:814: UserWarning: Neither SQLALCHEMY_DATABASE_URI nor SQLALCHEMY_BINDS is set. Defaulting S
QLALCHEMY_DATABASE_URI to "sqlite:///:memory:".
  'Neither SQLALCHEMY_DATABASE_URI nor SQLALCHEMY_BINDS is set. '
[2026-08-24 12:32:41,047] {manager.py:710} WARNING - No user yet created, use flask fab command to do it.
[2026-08-24 12:32:41,821] {__init__.py:51} INFO - Using executor CeleryExecutor
[2026-08-24 12:32:41,822] {dagbag.py:396} INFO - Filling up the DagBag from /usr/local/airflow/dags
[2026-08-24 12:32:43,084] {security.py:475} INFO - Start syncing user roles.
[2026-08-24 12:32:43,142] {security.py:207} INFO - Initializing permissions for role:Viewer in the database.
[2026-08-24 12:32:43,337] {security.py:207} INFO - Initializing permissions for role:User in the database.
[2026-08-24 12:32:43,471] {security.py:207} INFO - Initializing permissions for role:Op in the database.
[2026-08-24 12:32:43,600] {security.py:385} INFO - Fetching a set of all permission, view_menu from FAB meta-table
[2026-08-24 12:32:43,725] {security.py:328} INFO - Cleaning faulty perms
```

Id	Username	Email	First name	Last name	Roles

```
airflow@65409b16ec66:~$ airflow create_user -r Admin --username admin --password admin --email airflow@airflow.com --firstname matheus
Required arguments are missing: lastname.
airflow@65409b16ec66:~$ airflow create_user -r Admin --username admin --password admin --email airflow@airflow.com --firstname matheus --lastname beuren
/usr/local/lib/python3.7/site-packages/flask_sqlalchemy/__init__.py:814: UserWarning: Neither SQLALCHEMY_DATABASE_URI nor SQLALCHEMY_BINDS is set. Defaulting S
QLALCHEMY_DATABASE_URI to "sqlite:///:memory:".
  'Neither SQLALCHEMY_DATABASE_URI nor SQLALCHEMY_BINDS is set. '
[2026-08-24 12:35:56,826] {manager.py:710} WARNING - No user yet created, use flask fab command to do it.
[2026-08-24 12:35:57,769] {__init__.py:51} INFO - Using executor CeleryExecutor
[2026-08-24 12:35:57,769] {dagbag.py:396} INFO - Filling up the DagBag from /usr/local/airflow/dags
[2026-08-24 12:35:59,719] {security.py:475} INFO - Start syncing user roles.
[2026-08-24 12:35:59,842] {security.py:207} INFO - Initializing permissions for role:Viewer in the database.
[2026-08-24 12:36:00,019] {security.py:207} INFO - Initializing permissions for role:User in the database.
[2026-08-24 12:36:00,254] {security.py:207} INFO - Initializing permissions for role:Op in the database.
[2026-08-24 12:36:00,364] {security.py:385} INFO - Fetching a set of all permission, view_menu from FAB meta-table
[2026-08-24 12:36:00,497] {security.py:328} INFO - Cleaning faulty perms
Admin user admin created.
```

Figura 12 - Criação via UI



3. Conclusão

O Airflow resolve o problema da automatização dos processos, por meio dos dags que possuem tasks e essas tasks podem ser usadas de diversas formas, como por exemplo um pipeline de dados onde é realizado a coleta, o tratamento e treino de um modelo. Isso automatiza o processo manual que invés do acionamento manual de um script, agora é automático via airflow.